

claude-windows / 577fd4df-ed3f-43d3-854d-d63d732e2e07

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\577fd4df-ed3f-43d3-854d-d63d732e2e07\subagents\workflows\wf_e5e25aeb-1c0\agent-a8371e40f81cd7512.jsonl`
- SHA-256: `cc860dad038f7053cebc177b5e34b5b07d39b5b9bdd89881782bdeb5d4aa0919`
- Source modified: `2026-06-20T16:38:10+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_e5e25aeb-1c0`
- session_id: `577fd4df-ed3f-43d3-854d-d63d732e2e07`
```

Transcript

1. user (2026-06-20T16:36:28.347Z)

You are reviewing a NEW feature (P3) in a TypeScript SPA: a DETERMINISTIC, dependency-free query compiler that turns plain text ("over 10s", "top 15 longest", "at least 8 shots") into a filter+sort+limit selection over detected badminton rallies. NO LLM, NO network.

Read these files with the Read tool before judging:

```
- CORE parser:      C:/Users/avidu/Projects/khelsutra/web/src/lib/query.ts
- its unit tests:   C:/Users/avidu/Projects/khelsutra/web/src/lib/query.test.ts
- QueryBar component: C:/Users/avidu/Projects/khelsutra/web/src/components/QueryBar.tsx
- App wiring:       C:/Users/avidu/Projects/khelsutra/web/src/App.tsx
- selection hook:   C:/Users/avidu/Projects/khelsutra/web/src/hooks/useRallySelection.ts
- RallyGrid:        C:/Users/avidu/Projects/khelsutra/web/src/components/RallyGrid.tsx
- engine data types: C:/Users/avidu/Projects/khelsutra/web/src/api/types.ts
- the DESIGN CONTRACT: C:/Users/avidu/Projects/khelsutra/docs/PER_RALLY_SELECTION.md (read $2 decisions D5, $3 honest-vs-fabricated fields, $4.3 "Dead-ends / do-NOT", $9 DoD)
```

Report ONLY concrete, defensible findings with exact evidence (an input string ? the wrong parsed output, or a code line). Do NOT invent issues to fill a quota; an empty findings list is a valid, respected answer. Prefer a few high-confidence findings over many speculative ones.

Independently VERIFY this single finding by reading the actual code (do not trust the claim). Reproduce it concretely ? trace the exact input through the real regexes/logic, or point to the exact line. Set is_real=true ONLY if you confirm it. If it's a false positive or already handled, set is_real=false and explain why. Be a skeptic; default to is_real=false when uncertain.

FINDING (dimension grammar):

```
{
  "title": "\"first N seconds\" mis-parses as a LIMIT of N rallies, not a duration concept; the 'seconds' unit is ignored",
  "severity": "major",
  "file": "C:/Users/avidu/Projects/khelsutra/web/src/lib/query.ts",
  "evidence": "parseQuery(\"first 30 seconds\") ? {sort:{key:'start_time',dir:'asc'}, limit:30, filters:[]}. The limit branch on line 168, working.match(/\\b(?:first|earliest)\\s+(\\d+)\\b/), fires on 'first 30' before the trailing 'seconds' unit is ever considered, so '30' becomes a count of rallies.",
  "why": "A coach typing 'first 30 seconds' is expressing a time concept (the first 30s of the session, or rallies ?30s), not 'give me the 30 earliest rallies'. The bare-Ns rule deliberately refuses '30s' as a filter, yet 'first 30 seconds' silently becomes limit=30 ? a confidently wrong selection (caps to 30 cards) with a parsed-pill that reads 'oldest first · top 30', hiding the misread.",
  "suggested_fix": "Guard the first/earliest/last limit regexes so a following time/shot unit blocks the ordinal reading, e.g.
```

```

/\\b(?:first|earliest)\\s+(\\d+)(?!\\s*(?:s|secs?|seconds?|m|min|mins?|minutes?|shots?)\\b)/.
Then 'first 30 seconds' has no recognized actionable clause (consistent with the bare-'30s'
rule) instead of a fabricated limit.",
  "confidence": "high"
}

```

2. assistant (2026-06-20T16:36:34.127Z)

I'll verify this finding by reading the actual parser code and tracing the input through the real logic.

3. user (2026-06-20T16:36:34.595Z)

```

1      // Deterministic, dependency-free query compiler for the rally picker
(PER_RALLY_SELECTION.md
2      // D2/D5). NO LLM, NO network ? a tiny regex grammar over the ONLY honest, queryable
axes the
3      // engine returns today: rally LENGTH (duration), ORDER (start_time / duration /
shot_count),
4      // COUNT (top/first N), and SHOT COUNT (metadata.shot_count).
5      //
6      // Everything else a user might type ? shot TYPE (smash/clear/drop?), PLAYER, SCORE ?
has no
7      // detector and no column, so we FLAG it as "not available yet" and NEVER fabricate a
result.
8      // Parsing is pure and unit-tested; the QueryBar React component is a thin shell over
it.
9      import type { RallySegment } from "../api/types";
10
11     export type CompareOp = ">" | ">=" | "<" | "<=" | "=";
12     export type FilterField = "duration" | "shot_count";
13     /** One ANDed predicate over a rally field (e.g. duration > 10s). */
14     export interface Filter {
15       field: FilterField;
16       op: CompareOp;
17       value: number; // seconds for duration; a raw count for shot_count
18     }
19
20     export type SortKey = "duration" | "start_time" | "shot_count";
21     export interface Sort {
22       key: SortKey;
23       dir: "asc" | "desc";
24     }
25
26     /** A recognized-but-unsupported concept ? surfaced as a greyed "not available yet"
hint. */
27     export type UnavailableKind = "shot-type" | "player" | "score";
28     export interface Unavailable {
29       token: string;
30       kind: UnavailableKind;
31     }
32
33     export interface ParsedQuery {
34       raw: string;
35       /** ANDed together. */
36       filters: Filter[];
37       sort?: Sort;
38       /** "top N" / "first N" / "N longest". */
39       limit?: number;
40       /** Concepts we understood but cannot honour (shot-type / player / score). */
41       unavailable: Unavailable[];
42       /** True when at least one actionable clause (filter | sort | limit) was understood.
*/
43       recognized: boolean;
44     }

```

```

46 export interface QueryResult {
47     /** ALL rallies in display order (sorted if a sort was given, else input order). */
48     ordered: RallySegment[];
49     /** Ids matching the filters (then limited), in display order. */
50     selectedIds: number[];
51 }
52
53 // ?? grammar fragments ??????????????????????????????????????????????????????????????
54 const NUM = "(\\d+(?:\\.\\d+)?)";
55 // Optional trailing unit/noun. "shots?" is listed FIRST so it can't be eaten by the
bare "s"
56 // time-unit alternative; \b stops "m" matching "minutes" etc.
57 const UNIT = "(?:\\s*(shots?|seconds?|secs?|minutes?|mins?|min|m|s)\\b)?";
58
59 interface OpPattern {
60     op: CompareOp;
61     re: RegExp;
62     /** Which capture group holds the number / the unit. */
63     numAt: number;
64     unitAt: number;
65 }
66
67 // Leading comparator ("over 10s"). The "no more than" / "at least" guards come BEFORE
their
68 // bare counterparts ("more than" / "less than") so the negated phrase is consumed
first.
69 function lead(alt: string, op: CompareOp): OpPattern {
70     return { op, re: new RegExp(`(?:${alt})\\s*${NUM}${UNIT}`, "g"), numAt: 1, unitAt: 2
};
71 }
72 const LEADING: OpPattern[] = [
73     lead("no more than|at most|maximimum(?: of)?|max|up to|<=", "<="),
74     lead("no less than|no fewer than|at least|minimimum(?: of)?|min|>=", ">="),
75     lead("exactly|equal to>equals", "="),
76     lead("over|longer than|more than|greater than|above|bigger than|>", ">"),
77     lead("under|shorter than|less than|fewer than|below|smaller than|<", "<"),
78 ];
79
80 // Trailing comparator. The unit may sit BEFORE the operator ("10s+", "8 shots or more")
or the
81 // "shots" noun AFTER it ("10+ shots") ? capture both sides and use whichever is
present.
82 interface TrailPattern {
83     op: CompareOp;
84     re: RegExp;
85 }
86 const POST_NOUN = "(?:\\s*(shots?)\\b)?";
87 const TRAILING: TrailPattern[] = [
88     { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
(?:more|longer|over|above|greater)) ${POST_NOUN}`, "g") },
89     { op: "<=", re: new RegExp(`${NUM}${UNIT}\\s*or
(?:less|fewer|shorter|under|below) ${POST_NOUN}`, "g") },
90 ];
91
92 function toSeconds(n: number, unit?: string): number {
93     return unit && /^m/.test(unit) ? n * 60 : n; // m/min/minute(s) ? seconds; else as-is
94 }
95
96 function makeFilter(numStr: string, unit: string | undefined, op: CompareOp): Filter {
97     const n = Number(numStr);
98     const isShots = !!unit && /^shots?$/.test(unit);
99     return isShots
100         ? { field: "shot_count", op, value: n }
101         : { field: "duration", op, value: toSeconds(n, unit) };

```

```

102     }
103
104     const SORT_PATTERNS: Array<RegExp, Sort> = [
105         [/b(?:longest(?: first)?|longer first|by(?:duration|length)|duration desc)\b/, {
106             key: "duration", dir: "desc" }],
107         [/b(?:shortest(?: first)?|shorter first|duration asc)\b/, { key: "duration", dir:
108             "asc" }],
109         [/b(?:newest(?: first)?|latest|most recent|recent first)\b/, { key: "start_time",
110             dir: "desc" }],
111         [/b(?:oldest(?: first)?|earliest|chronological|in order|by time)\b/, { key:
112             "start_time", dir: "asc" }],
113         [/b(?:most shots(?: first)?|highest shots?|by shots?|by shot count|shot count
114             desc)\b/, { key: "shot_count", dir: "desc" }],
115         [/b(?:fewest shots(?: first)?|least shots?|lowest shots?|shot count asc)\b/, { key:
116             "shot_count", dir: "asc" }],
117     ];
118
119     // Order before kind so a token claimed by an earlier kind isn't double-counted.
120     const UNAVAILABLE_PATTERNS: Array<UnavailableKind, RegExp> = [
121         ["shot-type", /\b(smash(?:es)?|clears?|drops?|dropshots?|net
122             ?shots?|drives?|lifts?|serves?|kills?|taps?|pushes?|blocks?|slices?)\b/g],
123         ["player", /\b(players?|opponents?|servers?|receivers?)\b/g],
124         ["score", /\b(points?|score|deuce|game point|set point|match point)\b/g],
125     ];
126
127     // ?? public API ?????????????????????????????????????????????????????????????
128
129     export function parseQuery(raw: string): ParsedQuery {
130         const normalized = raw
131             .toLowerCase()
132             .replace(/>/g, ">=")
133             .replace(/</g, "<=")
134             .replace(/,/g, " ")
135             .replace(/\\s+/g, " ")
136             .trim();
137
138         const filters: Filter[] = [];
139         let working = normalized;
140
141         // Extract comparator clauses, blanking each match so later passes can't re-read its
142         // number.
143         for (const pat of LEADING) {
144             working = working.replace(pat.re, (full, ...g) => {
145                 filters.push(makeFilter(g[pat.numAt - 1], g[pat.unitAt - 1], pat.op));
146                 return " ".repeat(full.length);
147             });
148         }
149         for (const pat of TRAILING) {
150             working = working.replace(pat.re, (full, num, preUnit, postNoun) => {
151                 filters.push(makeFilter(num, preUnit ?? postNoun, pat.op));
152                 return " ".repeat(full.length);
153             });
154         }
155
156         // Explicit sort (runs before limit so "top 15 longest" keeps the longest?desc
157         // intent).
158         let sort: Sort | undefined;
159         for (const [re, s] of SORT_PATTERNS) {
160             if (re.test(working)) {
161                 sort = s;
162                 break;
163             }
164         }
165
166         // Limit ("top/best N", "first N", "last N", "N longest/shortest", "longest/shortest

```

```

N").
158     let limit: number | undefined;
159     let m: RegExpMatchArray | null;
160     if ((m = working.match(/\b(\d+)\s+(longest|shortest)\b/)) || (m =
working.match(/\b(longest|shortest)\s+(\d+)\b/))) {
161         const num = /^\\d/.test(m[1]) ? m[1] : m[2];
162         const word = /^\\d/.test(m[1]) ? m[2] : m[1];
163         limit = parseInt(num, 10);
164         if (!sort) sort = { key: "duration", dir: word === "longest" ? "desc" : "asc" };
165     } else if ((m = working.match(/\b(?:top|best)\s+(\d+)\b/))) {
166         limit = parseInt(m[1], 10);
167         if (!sort) sort = { key: "duration", dir: "desc" }; // "top N" ? best = longest
168     } else if ((m = working.match(/\b(?:first|earliest)\s+(\d+)\b/))) {
169         limit = parseInt(m[1], 10);
170         if (!sort) sort = { key: "start_time", dir: "asc" };
171     } else if ((m = working.match(/\b(?:last|latest|newest|most recent)\s+(\d+)\b/))) {
172         limit = parseInt(m[1], 10);
173         if (!sort) sort = { key: "start_time", dir: "desc" };
174     }
175
176     // Unavailable concepts (scanned on the normalized text; deduped, first kind wins).
177     const unavailable: Unavailable[] = [];
178     const seen = new Set<string>();
179     for (const [kind, re] of UNAVAILABLE_PATTERNS) {
180         for (const mm of normalized.matchAll(re)) {
181             const token = mm[1];
182             if (!seen.has(token)) {
183                 seen.add(token);
184                 unavailable.push({ token, kind });
185             }
186         }
187     }
188
189     return {
190         raw,
191         filters,
192         sort,
193         limit,
194         unavailable,
195         recognized: filters.length > 0 || sort != null || limit != null,
196     };
197 }
198
199 function rallyValue(r: RallySegment, key: SortKey): number | undefined {
200     if (key === "duration") return r.duration;
201     if (key === "start_time") return r.start_time;
202     return r.metadata?.shot_count;
203 }
204
205 /** Stable sort; rallies missing the sort field always sort LAST (unknown ? smallest).
*/
206 export function sortRallies(rallies: RallySegment[], sort: Sort): RallySegment[] {
207     return rallies
208         .map((r, i) => ({ r, i }))
209         .sort((a, b) => {
210             const av = rallyValue(a.r, sort.key);
211             const bv = rallyValue(b.r, sort.key);
212             if (av == null && bv == null) return a.i - b.i;
213             if (av == null) return 1;
214             if (bv == null) return -1;
215             if (av !== bv) return sort.dir === "asc" ? av - bv : bv - av;
216             return a.i - b.i; // tie ? preserve input order
217         })
218         .map((x) => x.r);
219 }

```

```

220
221 function matchesFilter(r: RallySegment, f: Filter): boolean {
222     const v = f.field === "duration" ? r.duration : r.metadata?.shot_count;
223     if (v == null) return false; // a rally without shot_count can't satisfy a shot filter
224     switch (f.op) {
225         case ">":
226             return v > f.value;
227         case ">=":
228             return v >= f.value;
229         case "<":
230             return v < f.value;
231         case "<=":
232             return v <= f.value;
233         case "=":
234             return v === f.value;
235         default:
236             return false;
237     }
238 }
239
240 /** Apply a parsed query to the rallies: ordered display list + the ids it selects.
Pure.
241     * An UNRECOGNIZED query (nothing understood) selects nothing and leaves order untouched
?
242     * so garbage input can never silently wipe a user's manual curation. The App also
guards on
243     * `q.recognized` before applying, so an empty box is a no-op end to end. */
244 export function runQuery(rallies: RallySegment[], q: ParsedQuery): QueryResult {
245     if (!q.recognized) return { ordered: rallies.slice(), selectedIds: [] };
246     const ordered = q.sort ? sortRallies(rallies, q.sort) : rallies.slice();
247     let matched = ordered.filter((r) => q.filters.every((f) => matchesFilter(r, f)));
248     if (q.limit != null) matched = matched.slice(0, q.limit);
249     return { ordered, selectedIds: matched.map((r) => r.id) };
250 }
251
252 // ?? human-readable labels (shared by the QueryBar pill and the tests) ?????????
253 const OP_SYMBOL: Record<CompareOp, string> = { ">": ">", ">=": "?", "<": "<", "<=": "?",
"=": "=" };
254
255 export function filterLabel(f: Filter): string {
256     if (f.field === "duration") return `length ${OP_SYMBOL[f.op]} ${f.value}s`;
257     return `shots ${OP_SYMBOL[f.op]} ${f.value}`;
258 }
259
260 export function sortLabel(s: Sort): string {
261     if (s.key === "duration") return s.dir === "desc" ? "longest first" : "shortest
first";
262     if (s.key === "start_time") return s.dir === "desc" ? "newest first" : "oldest first";
263     return s.dir === "desc" ? "most shots first" : "fewest shots first";
264 }
265
266 /** Human clause labels for the parsed-pill (e.g. ["length > 10s", "longest first", "top
15"]). */
267 export function summarize(q: ParsedQuery): string[] {
268     const parts: string[] = [];
269     for (const f of q.filters) parts.push(filterLabel(f));
270     if (q.sort) parts.push(sortLabel(q.sort));
271     if (q.limit != null) parts.push(`top ${q.limit}`);
272     return parts;
273 }
274

```

4. assistant (2026-06-20T16:36:38.084Z)

Now let me trace the exact input. I need to check what regexes touch "first 30 seconds". Let me

also read the test file and the design contract to confirm intended behavior.

5. user (2026-06-20T16:36:38.571Z)

```
1     import { describe, it, expect } from "vitest";
2     import {
3       parseQuery,
4       runQuery,
5       sortRallies,
6       filterLabel,
7       sortLabel,
8       summarize,
9     } from "../query";
10    import type { RallySegment } from "../api/types";
11
12    const mk = (
13      id: number,
14      start_time: number,
15      duration: number,
16      shot_count?: number,
17    ): RallySegment => ({
18      id,
19      start_time,
20      end_time: start_time + duration,
21      duration,
22      server: null,
23      receiver: null,
24      metadata: shot_count == null ? {} : { shot_count },
25    });
26
27    // A small, varied corpus: ids ordered chronologically, durations & shot counts mixed,
28    // rally 4 deliberately has NO shot_count (tests "unknown sorts/filters honestly").
29    const RALLIES: RallySegment[] = [
30      mk(1, 12, 8, 9),
31      mk(2, 41, 6, 5),
32      mk(3, 63, 21, 19),
33      mk(4, 95, 6 /* no shot_count */),
34      mk(5, 130, 12, 11),
35      mk(6, 158, 29, 24),
36    ];
37
38    describe("parseQuery ? duration filters", () => {
39      it("'over 10s' ? duration > 10", () => {
40        expect(parseQuery("over 10s").filters).toEqual([
41          { field: "duration", op: ">", value: 10 }
42        ]);
43      });
44      it("accepts varied units and phrasings", () => {
45        expect(parseQuery("longer than 10 seconds").filters[0]).toEqual({
46          field: "duration", op: ">", value: 10
47        });
48        expect(parseQuery("under 5 sec").filters[0]).toEqual({
49          field: "duration", op: "<", value: 5
50        });
51        expect(parseQuery("at least 8s").filters[0]).toEqual({
52          field: "duration", op: ">=", value: 8
53        });
54        expect(parseQuery("at most 8 seconds").filters[0]).toEqual({
55          field: "duration", op: "<=", value: 8
56        });
57        expect(parseQuery("exactly 7s").filters[0]).toEqual({
58          field: "duration", op: "=", value: 7
59        });
60      });
61      it("converts minutes to seconds", () => {
62        expect(parseQuery("over 1 minute").filters[0]).toEqual({
63          field: "duration", op: ">", value: 60
64        });
65        expect(parseQuery("at least 1.5m").filters[0]).toEqual({
66          field: "duration", op: ">=", value: 90
67        });
68      });
69      it("handles symbols and trailing comparators", () => {
```

```

54     expect(parseQuery("? 10s").filters[0]).toEqual({ field: "duration", op: ">=", value:
10 });
55     expect(parseQuery("> 10").filters[0]).toEqual({ field: "duration", op: ">", value:
10 });
56     expect(parseQuery("10s+").filters[0]).toEqual({ field: "duration", op: ">=", value:
10 });
57     expect(parseQuery("10s or longer").filters[0]).toEqual({ field: "duration", op:
">=", value: 10 });
58     expect(parseQuery("8s or less").filters[0]).toEqual({ field: "duration", op: "<=",
value: 8 });
59     });
60     it("does NOT consume 'no more than' as 'more than'", () => {
61         expect(parseQuery("no more than 5s").filters[0]).toEqual({ field: "duration", op:
"<=", value: 5 });
62     });
63     it("treats a bare 'Ns' (no comparator) as not a filter", () => {
64         expect(parseQuery("10s").filters).toEqual([]);
65         expect(parseQuery("10s").recognized).toBe(false);
66     });
67 });
68
69 describe("parseQuery ? shot_count filters (noun-keyed)", () => {
70     it("routes the 'shots' noun to shot_count, not duration", () => {
71         expect(parseQuery("over 10 shots").filters[0]).toEqual({ field: "shot_count", op:
">", value: 10 });
72         expect(parseQuery("at least 8 shots").filters[0]).toEqual({ field: "shot_count", op:
">=", value: 8 });
73         expect(parseQuery("fewer than 5 shots").filters[0]).toEqual({ field: "shot_count",
op: "<", value: 5 });
74         expect(parseQuery("10+ shots").filters[0]).toEqual({ field: "shot_count", op: ">=",
value: 10 });
75     });
76 });
77
78 describe("parseQuery ? sort", () => {
79     const cases: Array<[string, ReturnType<typeof parseQuery>["sort"]]> = [
80         ["longest first", { key: "duration", dir: "desc" }],
81         ["shortest", { key: "duration", dir: "asc" }],
82         ["newest first", { key: "start_time", dir: "desc" }],
83         ["oldest first", { key: "start_time", dir: "asc" }],
84         ["in order", { key: "start_time", dir: "asc" }],
85         ["most shots", { key: "shot_count", dir: "desc" }],
86         ["fewest shots", { key: "shot_count", dir: "asc" }],
87     ];
88     it.each(cases)("%s' ? %o", (input, expected) => {
89         expect(parseQuery(input).sort).toEqual(expected);
90     });
91 });
92
93 describe("parseQuery ? limit (and its implied sort)", () => {
94     it("'top 15' ? limit 15, sort longest-first", () => {
95         const q = parseQuery("top 15");
96         expect(q.limit).toBe(15);
97         expect(q.sort).toEqual({ key: "duration", dir: "desc" });
98     });
99     it("'first 10' ? limit 10, sort oldest-first", () => {
100         const q = parseQuery("first 10");
101         expect(q.limit).toBe(10);
102         expect(q.sort).toEqual({ key: "start_time", dir: "asc" });
103     });
104     it("'15 longest' / '5 shortest' carry both limit and sort", () => {
105         expect(parseQuery("15 longest")).toMatchObject({ limit: 15, sort: { key: "duration",
dir: "desc" } });
106         expect(parseQuery("5 shortest")).toMatchObject({ limit: 5, sort: { key: "duration",
dir: "asc" } });

```



```

107     });
108     it("an explicit sort wins over the limit's implied one ('top 15 longest')", () => {
109         const q = parseQuery("top 15 longest");
110         expect(q.limit).toBe(15);
111         expect(q.sort).toEqual({ key: "duration", dir: "desc" });
112     });
113     it("combines a filter with a limit ('over 10s top 5')", () => {
114         const q = parseQuery("over 10s top 5");
115         expect(q.filters).toEqual([{ field: "duration", op: ">", value: 10 }]);
116         expect(q.limit).toBe(5);
117     });
118 });
119
120 describe("parseQuery ? unavailable concepts are flagged, never faked", () => {
121     it("flags shot-type words while still applying the available clause", () => {
122         const q = parseQuery("smashes over 10s");
123         expect(q.filters).toEqual([{ field: "duration", op: ">", value: 10 }]); // honoured
124         expect(q.unavailable).toEqual([{ token: "smashes", kind: "shot-type" }]); // flagged
125     });
126     it("flags player and score concepts", () => {
127         expect(parseQuery("opponent rallies").unavailable).toContainEqual({ token:
"opponent", kind: "player" });
128         expect(parseQuery("game point").unavailable).toContainEqual({ token: "game point",
kind: "score" });
129     });
130     it("does not confuse 'shots' (count) with a shot-type", () => {
131         expect(parseQuery("over 10 shots").unavailable).toEqual([]);
132     });
133     it("a pure unavailable query is not recognized as actionable", () => {
134         const q = parseQuery("rallies with drops");
135         expect(q.recognized).toBe(false);
136         expect(q.unavailable).toContainEqual({ token: "drops", kind: "shot-type" });
137     });
138 });
139
140 describe("parseQuery ? unrecognized / empty input", () => {
141     it("returns recognized=false with nothing to apply", () => {
142         for (const s of ["", " ", "hello there", "asdf"]) {
143             const q = parseQuery(s);
144             expect(q.recognized).toBe(false);
145             expect(q.filters).toEqual([]);
146             expect(q.sort).toBeUndefined();
147             expect(q.limit).toBeUndefined();
148         }
149     });
150 });
151
152 describe("sortRallies", () => {
153     it("sorts by duration desc / asc, stably", () => {
154         expect(sortRallies(RALLIES, { key: "duration", dir: "desc" })).map((r) =>
r.id).toEqual([6, 3, 5, 1, 2, 4]);
155         expect(sortRallies(RALLIES, { key: "duration", dir: "asc" })).map((r) =>
r.id).toEqual([2, 4, 1, 5, 3, 6]);
156     });
157     it("puts rallies with a missing field LAST regardless of direction", () => {
158         // rally 4 has no shot_count ? last whether asc or desc.
159         expect(sortRallies(RALLIES, { key: "shot_count", dir: "desc" })).map((r) =>
r.id).toEqual([6, 3, 5, 1, 2, 4]);
160         expect(sortRallies(RALLIES, { key: "shot_count", dir: "asc" })).map((r) =>
r.id).toEqual([2, 1, 5, 3, 6, 4]);
161     });
162     it("does not mutate the input array", () => {
163         const ids = RALLIES.map((r) => r.id);
164         sortRallies(RALLIES, { key: "duration", dir: "desc" });
165         expect(RALLIES.map((r) => r.id).toEqual(ids);

```

```

166     });
167   });
168
169   describe("runQuery ? end to end", () => {
170     it("selects every rally over 10s, in chronological order (no sort)", () => {
171       const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("over 10s"));
172       expect(ordered.map((r) => r.id)).toEqual([1, 2, 3, 4, 5, 6]); // unsorted ? input
order
173       expect(selectedIds).toEqual([3, 5, 6]); // 21s, 12s, 29s
174     });
175     it("'top 3 longest' sorts all and selects the 3 longest", () => {
176       const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("top 3 longest"));
177       expect(ordered.map((r) => r.id)).toEqual([6, 3, 5, 1, 2, 4]); // all, longest-first
178       expect(selectedIds).toEqual([6, 3, 5]);
179     });
180     it("a shot filter excludes rallies that have no shot_count", () => {
181       const { selectedIds } = runQuery(RALLIES, parseQuery("at least 10 shots"));
182       expect(selectedIds).toEqual([3, 5, 6]); // rally 4 (no shot_count) excluded
183     });
184     it("filter + limit interact (longest 2 of the >10s rallies)", () => {
185       const { selectedIds } = runQuery(RALLIES, parseQuery("over 10s top 2"));
186       expect(selectedIds).toEqual([6, 3]); // sorted longest-first then capped at 2
187     });
188     it("an unrecognized query selects nothing and leaves order untouched", () => {
189       const { ordered, selectedIds } = runQuery(RALLIES, parseQuery("smashes only"));
190       expect(selectedIds).toEqual([]);
191       expect(ordered.map((r) => r.id)).toEqual([1, 2, 3, 4, 5, 6]);
192     });
193   });
194
195   describe("labels", () => {
196     it("filterLabel renders honest field + symbol + unit", () => {
197       expect(filterLabel({ field: "duration", op: ">", value: 10 })).toBe("length > 10s");
198       expect(filterLabel({ field: "shot_count", op: ">=", value: 8 })).toBe("shots ? 8");
199     });
200     it("sortLabel reads naturally for every key/dir", () => {
201       expect(sortLabel({ key: "duration", dir: "desc" })).toBe("longest first");
202       expect(sortLabel({ key: "duration", dir: "asc" })).toBe("shortest first");
203       expect(sortLabel({ key: "start_time", dir: "desc" })).toBe("newest first");
204       expect(sortLabel({ key: "start_time", dir: "asc" })).toBe("oldest first");
205       expect(sortLabel({ key: "shot_count", dir: "desc" })).toBe("most shots first");
206       expect(sortLabel({ key: "shot_count", dir: "asc" })).toBe("fewest shots first");
207     });
208     it("summarize lists filter, sort and limit clauses", () => {
209       expect(summarize(parseQuery("over 10s top 5"))).toEqual(["length > 10s", "longest
first", "top 5"]);
210       expect(summarize(parseQuery("hello"))).toEqual([]);
211     });
212   });
213

```

6. assistant (2026-06-20T16:36:46.481Z)

Now let me trace `parseQuery("first 30 seconds")` through the actual code step by step.

****Normalization:**** ` "first 30 seconds" ` ? lowercase, no commas, collapse whitespace ? ` "first 30 seconds" ` (unchanged).

****LEADING patterns:**** None match ? "first", "30", "seconds" don't contain any leading comparator like over/under/at least. So `filters` stays empty, `working` = "first 30 seconds".

****TRAILING patterns:**** Need `NUM UNIT (+ | or more...)`. No "+" or "or more" here. No match. `working` unchanged.

****SORT_PATTERNS:**** Check each against ` "first 30 seconds" `:

- longest/shortest/newest/latest ? no
- `oldest first|earliest|chronological|in order|by time` ? "earliest" no, none match. Wait ? is "first" matched? The pattern is `oldest(?: first)?` that requires "oldest" then optional "first". "first" alone is not matched. No sort.

****LIMIT branch:**** Line 168 `working.match(/\b(?:first|earliest)\s+(\d+)\b/)` against `"first 30 seconds"`. This matches `"first 30"` ? `\b(\d+)\b` matches "30" (the `\b` after `\d+` sits between "30" and the space before "seconds"). So `limit = 30`, and since no sort, `sort = {key:'start\_time', dir:'asc'}`.

Let me verify the regex with Node to be certain.

7. user (2026-06-20T16:36:48.351Z)

```
"first 30 seconds" -> limit=30  
"first 30s" -> no match  
"first 30 shots" -> limit=30  
"first 30 minutes" -> limit=30  
"first 10" -> limit=10
```

8. assistant (2026-06-20T16:36:53.652Z)

Interesting ? `"first 30s"` (no space) does NOT match because `\b(\d+)\b` requires a word boundary right after the digits, and "30s" has no boundary between "30" and "s". But `"first 30 seconds"` (with space) DOES match because the boundary falls between "30" and the space.

Now let me confirm the full `parseQuery` output by actually compiling and running the TS module. Let me check if there's a build setup, or I'll inline-transpile the relevant logic by importing via ts. Simpler: reproduce the full function path with Node by transpiling. Let me just run the whole parser through tsx/ts-node if available, otherwise replicate faithfully.

9. user (2026-06-20T16:36:58.440Z)

package.json

10. assistant (2026-06-20T16:37:02.111Z)

tsx isn't installed (no output / silent fail). Let me try the project's test runner approach ? transpile inline by stripping types is error-prone. Let me check what's available and try running via vitest with a throwaway test, which is the most faithful.

11. user (2026-06-20T16:37:04.020Z)

```
esbuild  
esbuild.cmd  
esbuild.ps1  
vitest  
vitest.cmd  
vitest.ps1  
---deps---  
{  
  "react":"^19.0.0",  
  "react-dom":"^19.0.0",  
  "@tailwindcss/vite":"^4.0.0",  
  "@types/react":"^19.0.0",  
  "@types/react-dom":"^19.0.0",  
  "@vitejs/plugin-react":"^4.3.4",  
  "tailwindcss":"^4.0.0",  
  "typescript":"^5.7.0",  
  "vite":"^6.0.0",  
  "vitest":"^3.0.0"  
}
```

12. assistant (2026-06-20T16:37:07.502Z)

esbuild is available ? I'll transpile the module and run it through Node faithfully.

13. user (2026-06-20T16:37:11.800Z)

```
Exit code 1  
node:internal/modules/run_main:107  
  triggerUncaughtException(  
    error: Error: Cannot find module 'esbuild' from 'node_modules/run_main'  )  
  ^  
Error: Cannot find module 'esbuild' from 'node_modules/run_main'
```

^

```
Error [ERR_MODULE_NOT_FOUND]: Cannot find module
'C:\Users\avidu\AppData\Local\Temp\node_modules\esbuild\lib\main.js' imported from
C:\Users\avidu\AppData\Local\Temp\probe.mjs
Did you mean to import "../../Projects/khelsutra/web/node_modules/esbuild/lib/main.js"?
    at finalizeResolution (node:internal/modules/esm/resolve:271:11)
    at moduleResolve (node:internal/modules/esm/resolve:865:10)
    at defaultResolve (node:internal/modules/esm/resolve:992:11)
    at #cachedDefaultResolve (node:internal/modules/esm/loader:694:20)
    at #resolveAndMaybeBlockOnLoaderThread (node:internal/modules/esm/loader:711:38)
    at ModuleLoader.resolveSync (node:internal/modules/esm/loader:743:52)
    at #resolve (node:internal/modules/esm/loader:676:17)
    at ModuleLoader.getOrCreateModuleJob (node:internal/modules/esm/loader:596:35)
    at ModuleJob.syncLink (node:internal/modules/esm/module_job:163:33)
    at ModuleJob.link (node:internal/modules/esm/module_job:253:17) {
  code: 'ERR_MODULE_NOT_FOUND',
  url: 'file:///C:/Users/avidu/AppData/Local/Temp/node_modules/esbuild/lib/main.js'
}
```

Node.js v26.1.0

14. assistant (2026-06-20T16:37:14.882Z)

The probe ran from /tmp. Let me write the probe into the project dir so relative imports resolve.

15. user (2026-06-20T16:37:18.543Z)

```
"first 30 seconds"
{"filters":[],"sort":{"key":"start_time","dir":"asc"},"limit":30,"recognized":true} pills=
["oldest first","top 30"]
"first 5 minutes"
{"filters":[],"sort":{"key":"start_time","dir":"asc"},"limit":5,"recognized":true} pills=
["oldest first","top 5"]
"earliest 20 seconds"
{"filters":[],"sort":{"key":"start_time","dir":"asc"},"limit":20,"recognized":true} pills=
["oldest first","top 20"]
"first 30s"           {"filters":[],"recognized":false} pills= []
"first 30 shots"
{"filters":[],"sort":{"key":"start_time","dir":"asc"},"limit":30,"recognized":true} pills=
["oldest first","top 30"]
"10s"                {"filters":[],"recognized":false} pills= []
"first 10"
{"filters":[],"sort":{"key":"start_time","dir":"asc"},"limit":10,"recognized":true} pills=
["oldest first","top 10"]
```

16. assistant (2026-06-20T16:37:27.715Z)

The finding is reproduced exactly against the real compiled code:

```
- `parseQuery("first 30 seconds")` ? `{filters:[], sort:{key:'start_time',dir:'asc'}, limit:30,
  recognized:true}`, pills `["oldest first","top 30"]`.
```

This confirms every claim in the finding:

1. The `seconds` unit is ignored ? "30" becomes a limit (count of rallies), not a duration concept.
2. The limit branch on line 168 (`/\b(?:first|earliest)\s+(\d+)\b/`) fires on "first 30" before the trailing "seconds" is considered.
3. The parsed pill reads "oldest first · top 30", hiding the misread.
4. Contrast with the bare-`30s` rule: `parseQuery("10s")` ? `recognized:false` (line 63-66 test), and indeed `"first 30s"` (no space) ? `recognized:false`, while `"first 30 seconds"` ? `limit:30`. So the same number written two ways gives wildly different results, and the unit-aware refusal is silently bypassed when a space separates digit and unit.

Now let me check the design contract to confirm this is genuinely a contract violation / undesired behavior, not an intended/documented edge case.

17. user (2026-06-20T16:37:28.177Z)

```
1      # Interactive per-rally selection ? reel builder (RallyPicker)
2
3      > **Status:** `IN PROGRESS` ? design **APPROVED by owner 2026-06-20**; building the MVP
(P1?P5). Downstream milestones (multi-video auth, per-rally preview endpoint, any `/api/compile`
default change) remain owner-gated. . **Owner:** Avi . **Created:** 2026-06-20 . **Last
updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6      > **Relation:** second major tracked work item in the `khelsutra` repo (peer of
`REQUEST_ACCESS_FORM.md`); mirrors the engine repo's `PROJECT_DOC_TEMPLATE` discipline.
Primarily realizes alpha screen **A5 Highlights** (engine repo `UI_PROPOSAL.md:47`, exec-plan
**M4** `UI_ALPHA_EXECUTION_PLAN.md:39`); its correction-loop extension realizes **A4 Rally
Review** (`UI_PROPOSAL.md:46`, exec-plan **M3** `:38`). Consumes the engine's `/api/query` +
`/api/compile` contract.
7      > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
code this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9      ---
10
11     ## 0. TL;DR
12     Build the **pick-rallies ? reel** surface as a **hybrid**: a selectable rally
**list/grid** is the workhorse; a **deterministic query bar** ("rallies over 10s, longest
first") pre-selects into the **same** selection set; a sticky footer compiles the selection into
one downloadable MP4. The entire MVP runs on **existing engine endpoints** ? `POST /api/query` ?
`POST /api/compile` ? `GET /api/highlights` `[verified backend/main.py:331-340, 342-396,
307-328]` ? with **zero new engine intelligence and zero new endpoints**. The single most
important caveat: queries and chips are scoped to **length / count / order / shot-count only**;
shot-type, player, and score are **roadmap-only** (no detector, no column) and must not be
faked. The biggest UX-vs-effort tension: **no per-rally preview is possible today** ? only the
**final compiled reel* can stream ? so MVP selects on text metadata, and visual preview/timeline
is a deliberate later milestone.
13
14     ## 1. Problem & goal
15     Today the alpha can detect rallies and stitch a reel, but reel-building is either a
**silent `stitching.min_shot_count` threshold** `[verified backend/main.py:362-383]` or a static
checkbox list whose data plumbing is even broken ? the bundled static UI reads `data.rallies`
while the API returns `play_segments`, so it renders nothing `[verified
backend/api/static/app.js:344 vs backend/main.py:340]`. The owner's framing is **dynamic per-
video selection, not a fixed threshold**: a coach should say "give me the long rallies from this
session" and get exactly those, then fine-tune by hand.
16
17     **The concrete outcome ("good"): A coach uploads a 40-minute session; the engine finds
60 rallies. They type **"top 15 longest", eyeball the auto-selected cards, untick two warm-up
rallies, name it **"Tuesday ? long points", and download a ~6-minute reel ? without learning any
threshold knob.
18
19     **Goal:** the smallest **honest** surface that turns engine-detected rallies into a user-
curated, downloadable reel, with both **fast** (query) and **precise** (checkbox) selection
sharing **one** state.
20
21     **Non-goals (alpha):** no conversational LLM (`UI_PROPOSAL.md:66` defers it); no shot-
type/player/score filters (no data exists); no per-rally video preview while it requires unbuilt
streaming; no multi-tenant auth (single-box alpha).
22
23     ## 2. Decisions locked
24     | # | Decision | Source / date | Implication |
25     |---|-----|-----|-----|
```

26 | D1 | Dominant surface = selectable rally ****list/grid**** | Synthesis 2026-06-20 (Grid scored 15, fit-to-data 4) | Scales to long matches via bulk+filter; shows honest fields only |

27 | D2 | Fast entry = ****deterministic**** client-side ``parseQuery()`` (no LLM) | ``[verified UI_PROPOSAL.md:66]`` conversational QA deferred | Predictable, unit-testable, offline, zero API cost |

28 | D3 | ****One**** shared ``Set<segment_id>``, mutated by query + checkbox + (future) timeline | Synthesis 2026-06-20 | No second compile path can diverge from what the user sees |

29 | D4 | Compile via existing ``POST /api/compile {video_id, segment_ids, output_filename}`` | ``[verified backend/main.py:342-396]`` | Selection order is harmless ? stitcher ``_merge_overlapping`` de-dupes ``[verified compiler.py:80-105, 311-317]`` |

30 | D5 | Query grammar (alpha) = length, count, order, ****shot_count only**** | ``[verified database.py:122-135, gemini.py:470-485]`` | Only fields the payload actually returns; grey out the rest |

31 | D6 | MVP backend = ****zero new endpoints**** | ``[verified]`` reuse map (§4) | Ships the full loop on the already-shipped contract |

32 | D7 | Replace the silent ``_min_shot_count`` cut with ****explicit user selection**** | Owner framing + ``[verified main.py:362-383]`` | UI ***warns*** about the floor; it never hides why a rally vanished |

33 | D8 | ****Default selection = ON**** (curate-by-removal) | Owner 2026-06-20 | Rallies load all-selected (matches today's "all rallies" reel); user removes warm-ups. Footer/empty UX assumes a non-empty start |

34 | D9 | ****Preview-before-select is NOT a ship-blocker**** for the MVP | Owner 2026-06-20 | MVP selects on text metadata + previews the ***final compiled reel***; the source/clip-stream endpoint (per-rally preview) is the next milestone (P9), not MVP |

35 | D10 | ****MVP is job-tethered**** (carry ``result.video_id``); no ``GET /api/videos`` | Owner 2026-06-20 | Smallest honest slice; multi-video home + identity scoping is P8 (gated) |

36 | D11 | ``_min_shot_count``: ****warn, do NOT override**** in the MVP | Owner 2026-06-20 | The UI surfaces the floor; an explicit-selection override of the compile default is deferred (would trigger the Launch Report convention) |

37

38 **## 3. Foundation ? what the engine gives us today** ``[verified this session]``

39 The whole design rests on the engine's real, already-shipped contract. Read against the engine repo (``badminton-highlight-indexer``):

40

41 - ****The rally store is ``play_segments`` ? columns ``id, video_id, start_time, end_time, duration, server, receiver, metadata`` ``[verified database.py:124-135]``. ``id`` is the integer the UI selects and passes to compile; ``start_time/end_time/duration`` are REAL seconds (duration computed at insert).**

42 - ****Listing rallies = ``POST /api/query {video_id}`` ? ``{play_segments:[...]}`` ? the ***only* rally-list path today; it JSON-parses ``metadata`` and joins ``events`` ``[verified main.py:331-340, database.py:427-475]``.****

43 - ****Building a reel = ``POST /api/compile {video_id, segment_ids:[int], output_filename}`` ? loads ``videos.filepath``, filters all segments to the requested ``segment_ids``, applies the ``_min_shot_count`` floor, extracts ``time_pairs=[(start,end)]``, and calls ``VideoCompiler.compile_highlights(raw_path, segments, name)`` ``[verified main.py:342-396, compiler.py:303]``, which ****merges overlaps**** then per-clip re-encodes ? concat ? one MP4 ``[verified compiler.py:311-317]``. Selection order/duplicates are therefore safe.**

44 - ****Downloading = ``GET /api/highlights/{video_id}/{filename}`` ? streams the compiled MP4. Compile returns a ****server-local filepath, NOT a URL**** ``[verified main.py:394]``, so the SPA constructs this download URL itself. The compiled reel is the ****only streamable video that exists today**** ``[verified main.py:307-328]``.**

45 - ****The floor knob is readable** ? ``GET /api/config`` exposes ``stitching.min_shot_count`` ``[verified main.py:121-123]``, so the UI can warn before a selected rally is silently dropped.**

46 - ****Job entry** ? ``GET /api/jobs/{job_id}`` returns ``result.video_id`` ``[verified main.py:274-293]``, which is how the MVP reaches a video without a (nonexistent) list-videos endpoint.**

47

48 ****Honest fields vs fabricated/roadmap (critical for the UI):****

49 - ****Honest today**** (default ``default_segmenter: "gemini"``, per ``config.json`` ``[verified]``): ``start_time``, ``end_time``, ``duration``, and ``metadata.shot_count`` (Gemini's ``shuttle_exchange_count``, fallback ``max(3, int(dur/0.8))``) + ``metadata.shot_count_confidence`` (a ****string****, often ``"Unknown"``) ``[verified gemini.py:470-485]``. On the gemini path ``server``/``receiver`` are written ``None`` ("no fabricated data") ``[verified gemini.py:480-487]``.

50 - ****Fabricated ? do NOT surface:**** the ``motion`` demo segmenter fills ``server``/``receiver`` via ``random.sample(...)``, a random ``shot_count``, and random ``events`` (serve/smash/foul/winner)

```

`[verified motion.py:191-198]`. `query` returns these columns + `events`, but they are not
ground truth and must stay hidden.
51 - **Roadmap ? no column, no detector:** shot-type per shot, player identity, match/point
score, game/set grouping. None exist anywhere in the schema; do not consume them `[verified
database.py whole-schema 122-148]`.
52 - **Correction-loop columns exist but are dead:**
`candidate_segments.human_label/notes/confirmed_segment_id` are defined but no endpoint writes
them `[verified database.py:166-168]` ? that's the M4 corpus extension (engine PR-4/B6,
`UI_ALPHA_EXECUTION_PLAN.md`).
53
54 ## 4. Design / architecture
55
56 **Reuse map (existing ? `[verified]`, no change for MVP):** `POST /api/query` (list) ?
`POST /api/compile` (stitch) ? `GET /api/highlights/{video_id}/{filename}` (download); `GET
/api/config` (floor) ; `GET /api/jobs/{id}` (carry `result.video_id`). **All net-new code is the
SPA** in `khelsutra` web/` (Vite + React + TS + Tailwind, per `web/README.md`), plus a small
set of *optional* real-gap engine enablers (§4.3) ? none required for MVP.
57
58 **4.1 Components (net-new SPA)**
59 - **QueryBuilder + `parseQuery()`** ? a *pure, deterministic, unit-tested* string ?
`{filter, sort}` compiler over `duration` / `ordinal` / `shot_count`. Renders a **parsed-pill**
("filter: duration > 10s · sort: duration desc") so the interpretation is transparent and hand-
correctable; greys out shot-type/player/score tokens with an inline "not available yet". No LLM,
no network call.
60 - **RallyList/Grid** ? one card per `play_segment`: ordinal (by `start_time`), `mm:ss`
start, duration (`end?start`), and a shot-count + confidence chip **only when
`metadata.shot_count` is present** (absent on detector paths that don't write it). Checkbox per
card; the thumbnail area is a placeholder (no endpoint yet).
61 - **`useRallySelection()` hook** ? owns `selectedIds:Set<number>`, `displayOrder`,
`apply(queryResult)`, `toggle(id)`, and bulk Select-all / Clear / Invert. Unit-tested (matches
the exec-plan's "API client + review-state logic" bar).
62 - **SelectionFooter (sticky)** ? live "N clips / total M:SS"; a **`min_shot_count`
warning** when a selected rally would be dropped at compile (read from `GET /api/config`); reel-
name input; one **Build** button.
63 - **CompileResult** ? `[` preview of the *compiled
reel* + a download link.
64 - **TypedApiClient** ? `query`, `compile`, `buildDownloadUrl`; the typed seam to the
engine.
65
66 **4.2 Data flow** `[verified anchors inline]`
67 Gemini segmenter (process time) writes `play_segments {id, start/end_time, duration,
metadata.shot_count/confidence/detector}` `[gemini.py:461-498]`
68 ? SPA `POST /api/query {video_id}` ? `{play_segments:[...]}` `[main.py:331-340]`
(**fix** the static UI's `data.rallies` read ? `play_segments` `[app.js:344]`)
69 ? normalize to `RallyCard[]` + one shared `selectedIds:Set`;
`server`/`receiver`/`events` are present in the payload but motion-only fabrications ? **not
surfaced** `[motion.py:191-198]`
70 ? all input modes mutate the **same** `selectedIds`: (a) `parseQuery()` predicate over
duration/ordinal/shot_count; (b) checkbox toggle + bulk ops; (c) *(post-MVP)* timeline band
71 ? footer recomputes count + ?duration **client-side**, and warns for any selected rally
with `shot_count < stitching.min_shot_count` `[main.py:362-383]`
72 ? **Build** ? `POST /api/compile {video_id, segment_ids: [...selectedIds],
output_filename}` ? stitcher de-dupes via `_merge_overlapping` `[compiler.py:311-317]`, cuts
`time_pairs` from `videos.filepath`, re-encodes per clip ? concat ? one MP4 `[main.py:386-393,
compiler.py:303]`
73 ? compile returns a **server-local filepath, not a URL** `[main.py:394]` ? SPA builds
`GET /api/highlights/{video_id}/{output_filename}` to preview in `` + download
`[main.py:307-328]`.
74
75 **4.3 Engine API additions (none for MVP; tagged against real gaps)**
76 | Endpoint | Purpose | Status |
77 |---|---|---|
78 | `POST /api/query` | List rallies (only path) ? fix SPA to read `play_segments` |
**exists** |
79 | `POST /api/compile` | Stitch selected `segment_ids` ? MP4 | **exists** |
](GET)
```

```

80      | `GET /api/highlights/{video_id}/{filename}` | Stream/download compiled reel |
**exists** |
81      | `GET /api/config` | Read `stitching.min_shot_count` for the warning | **exists** |
82      | `GET /api/jobs/{job_id}` | Carry `result.video_id` into the picker | **exists** |
83      | `POST /api/compile` ? add `download_url` | Return `/api/highlights/...` so SPA stops
hand-building it (additive, low-risk; today returns only a filepath `main.py:394`) | **modify**
|
84      | `GET /api/health` | Engine-offline banner ping the SPA is specified to use
(`HOSTING_PLAN.md:42`) | **new** |
85      | `GET /api/videos/{video_id}/rallies` | RESTful/cacheable alias returning the same
`{play_segments}` shape | **new** |
86      | `GET /api/videos` | List a user's videos for a Match picker (only `get_video(id)`
exists, `database.py:418`) ? needs DB enumerate + identity scoping | **new** |
87      | `GET /api/videos/{video_id}/source` (Range) *or* `/clip?start=&end=` | Stream
source/per-span video so a rally previews before selection ? neither exists | **new** |
88      | `GET /api/videos/{video_id}/rallies/{segment_id}/thumbnail` | Per-rally poster still
(no thumbnail column; WASB frames are transient) | **new** |
89      | `PATCH /api/segments/{id}` | Persist accept/reject/nudge ?
`candidate_segments.human_label/notes/...` (columns exist, unwritten ? corpus loop) | **new** |
90
91      **? Dead-ends / do-NOT:** never surface `server`/`receiver`/`events` (motion-only
randoms `[motion.py:191-198]`); never consume shot-type/player/score (no data); never render a
confidence **meter** ? `shot_count_confidence` is a Gemini *string* (often `Unknown`) and
`play_segments` has **no** numeric confidence column.
92
93      ## 5. Risk table
94      | Risk | Severity | Mitigation |
95      |---|---|---|
96      | **No source/clip/frame/thumbnail endpoint** ? only compiled reels stream
`[main.py:307-328]` | High (UX) | MVP selects on text metadata + previews the *final* reel;
preview/timeline deferred to M3 behind a new endpoint |
97      | `min_shot_count` silently drops selected rallies `[main.py:362-383]` | High (looks
broken) | Footer warns *before* Build; D7 makes selection explicit; an override is an owner gate
(Launch-Report-worthy `[memory: launch-report-convention]`) |
98      | No match-duration field ? a timeline must derive `total = max(end_time)`, mis-
rendering pre/post dead air | Med | Timeline deferred to M3; never over-promise it as a true
scrubber until source streams |
99      | `shot_count` is an *estimate* (`shuttle_exchange_count` w/ `max(3,int(dur/0.8))`
fallback) `[gemini.py:470-485]` | Med | Label "approx."; confidence shown as the raw string, not
a meter |
100     | No auth / user scoping anywhere (CORS ``, `videos.user_id` never set/filtered) | High
for multi-tester | Fine for single-box local alpha; a ship-blocker for any shared host ? gate
`GET /api/videos` behind PR-3 scoping |
101     | compile returns a filepath not a URL `[main.py:394]` | Low | Client builds the URL for
MVP; `download_url` field (M1) removes the brittleness |
102     | Surface sprawl (query + grid + future timeline + future correction) | Med | Deliberate
IA: query+grid = one screen; correction = separate ? corpus screen (owner call, $8) |
103
104     ## 6. Honest limits ? what this does NOT do
105     A green SPA build + passing `parseQuery`/selection unit tests prove the **picker logic
only** ? **not** that the engine's rally boundaries are correct (the measured **rally-END**
weakness is a separate quality track, unaffected by this UI). "Conversational" here means a
**deterministic four-dimension parser**, not natural-language understanding. Nothing is
implemented in `web/` yet ? this is `[design]`. The feature curates *what the engine already
found*; it cannot surface a rally the detector missed, nor split/merge a mis-bounded one (that's
the M4 correction loop).
106
107     ## 7. Deliverables & progress tracker ? **source of truth**
108     Legend: ? Todo . ? In progress . ? Done . ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. CI (`npm test`) must stay green; merge to `master`
auto-deploys the site (docs/`web/` changes don't touch the live `site/` Worker).
109
110     | ID | Deliverable | Depends on | Gated? | Status | PR |
111     |---|-----|-----|-----|-----|---|
112     | P0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | No | ? |

```



```
[#11](https://github.com/avidullu/khelsutra/pull/11) |
113 | P1 | SPA scaffold (Vite+React+TS+Tailwind) in `web/` + typed API client; dev proxy to
engine `:8000` | P0 | No | ? | [#14](https://github.com/avidullu/khelsutra/pull/14) |
114 | P2 | RallyList/Grid (honest fields only) + `useRallySelection()` hook (unit-tested) |
P1 | No | ? | [#21](https://github.com/avidullu/khelsutra/pull/21) |
115 | P3 | `parseQuery()` + parsed-pill + example chips (unit-tested; greys out unavailable
tokens) | P2 | No | ? | (this PR) |
116 | P4 | SelectionFooter + `min_shot_count` warning (from `GET /api/config`) | P2 | No | ?
| ? |
117 | P5 | Build ? `POST /api/compile` ? client-built download URL + `` preview of
the reel | P2 | No | ? | ? |
118 | P6 | *(engine repo)* `GET /api/health` + offline banner; `/api/compile` returns
`download_url` | ? | No | ? | ? |
119 | P7 | *(engine repo)* `GET /api/videos/{id}/rallies` RESTful alias | ? | No | ? | ? |
120 | P8 | *(engine repo)* `GET /api/videos` + PR-3 identity scoping ? A1 Matches screen |
P7 | ? owner | ? | ? |
121 | P9 | *(engine repo)* source/clip stream + timeline + per-rally preview/thumbnail | P8
| ? owner | ? | ? |
122 | P10 | *(engine repo)* `PATCH /api/segments/{id}` corpus loop (PR-4/B6) ? A4 Rally
Review | P9 | ? owner | ? | ? |
123
124 **Milestone rollout:** *M0/MVP = P1?P5** (pick?reel on existing API, zero new engine
endpoints) . *M1 = P6?P7** (RESTful + reliability polish) . *M2 = P8** (multi-video home) .
*M3 = P9** (per-rally preview/timeline) . *M4 = P10** (corpus correction loop) . *M5** = swap
`parseQuery()` for localized NL behind the same seam (gated on shot/player data landing).
125
126 ## 8. Open questions ? owner / external
127 **Blocking gates ? ? RESOLVED 2026-06-20 (owner) ? see D8?D11 in S2:**
128 1. ~~Default selection ON vs OFF?~~ ? **ON / curate-by-removal** (D8).
129 2. ~~Is preview-before-select a ship-blocker?~~ ? **No** ; text-metadata MVP, preview is
P9 (D9).
130 3. ~~Job-tethered vs multi-video MVP?~~ ? **Job-tethered** (D10).
131 4. ~~`min_shot_count` override?~~ ? **Warn, don't override** in MVP (D11).
132
133 **Still open (nice-to-knows, non-blocking):**
134 5. May user-facing copy call `parseQuery()` "conversational", given conversational QA is
explicitly deferred (UI_PROPOSAL.md:66)? Risk of over-promising NL. *Owner positioning/honesty
call.* *(Lean: avoid the word; show the parsed-pill instead.)*
135 6. One screen for query+grid (recommended) vs a separate ? corpus/correction screen for
A4? *Owner positioning call.*
136
137 ## 9. Definition of done
138 - [ ] From a finished job, a user can list rallies, query/select, and download a reel ?
on **existing** endpoints.
139 - [ ] Only honest fields (ordinal / duration / shot_count+confidence-when-present) are
shown; fabricated `server`/`receiver`/`events` are hidden.
140 - [x] `parseQuery()` covers length/count/order/shot_count, greys out unavailable tokens,
shows a visible parsed-pill, and is unit-tested. *(P3)*
141 - [ ] Selection hook is unit-tested; the `min_shot_count` floor is surfaced, not hidden.
142 - [ ] `npm run build` (and `npm test`) clean; the static UI's `data.rallies` bug is gone
(or the static UI is superseded by the SPA).
143 - [ ] S7 tracker + this DoD updated; doc archived when P1?P5 land (MVP).
144
145 ## 10. References
146 **Engine code anchors `[verified this session]`:** `backend/main.py:121-123` (config),
`:274-293` (jobs), `:307-328` (highlights download), `:331-340` (query), `:342-396` (compile +
`min_shot_count` floor + filepath-not-URL); `backend/infrastructure/database.py:124-135`
(play_segments), `:166-168` (dead correction columns), `:418` (no list-videos), `:427-475`
(query read path + events join); `backend/pipeline/segmenters/gemini.py:461-498, 470-485`
(honest shot_count, server/receiver=None); `backend/pipeline/segmenters/motion.py:191-198`
(fabricated fields); `backend/pipeline/stitcher/compiler.py:80-105, 303, 311-317` (merge/de-dupe
+ entypoint); `backend/api/static/app.js:344` (`data.rallies` bug); `config.json`
(`default_segmenter: "gemini")`.
147 **Engine docs:** `UI_PROPOSAL.md:46-47, 66`; `UI_ALPHA_EXECUTION_PLAN.md:38-39`;
`HOSTING_PLAN.md:42`; `OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55` (conversational seam built now,
```

```

model punted). **This repo:** `web/README.md`, `REQUEST_ACCESS_FORM.md`.
148
149     ---
150     ### Changelog
151     - 2026-06-20 ? Initial `DRAFT`. Consolidated three candidate designs (Timeline / Grid /
RallyPicker) into the hybrid recommendation via a multi-agent design pass; every engine-fact
anchor re-verified against source before writing.
152     - 2026-06-20 ? Owner **approved** the recommendations; locked D8?D11 (default ON ·
preview not a ship-blocker · job-tethered MVP · warn-don't-override the floor); status `DRAFT` ?
IN PROGRESS`; P0 done.
153     - 2026-06-20 ? **P3 shipped**: deterministic `parseQuery()` (`web/src/lib/query.ts`)
over length/count/order/shot_count + `runQuery`/`sortRallies`/label helpers (35 unit tests) and
the `QueryBar` (live parsed-pill, example chips, greyed "not available yet" for shot-
type/player/score ? never faked). Writes the shared selection set via a new
`useRallySelection().apply`; `RallyGrid` ordinals made stable so a re-sort doesn't renumber
cards. Live browser CUJ replayed (filter / top-N / honest-greying / untick / no-op). Suite 51
green, typecheck + build clean.
154

```

18. assistant (2026-06-20T16:37:38.237Z)

The contract is clear. The design philosophy (§0, §3, §4.1, D2/D5) is that the parser must be **honest and transparent** ? it parses over length/count/order/shot_count, and the parsed-pill must make "the interpretation transparent." The bare-`30s` test (line 63-66) deliberately refuses a bare duration with no comparator, treating it as not-a-filter.

The finding exposes a real inconsistency:

- `first 30s` ? `recognized:false` (no actionable clause) ? the unit-aware refusal kicks in via the `b` boundary mismatch.
- `first 30 seconds` ? `limit:30`, pill "oldest first · top 30" ? silently caps to 30 cards, ignoring the explicit time unit.

Same number, two spellings, two opposite outcomes. A coach typing "first 30 seconds" expresses a time concept, and the parser confidently returns a fabricated count-limit while the transparent pill claims "top 30," hiding the misread. This is exactly the "confidently wrong selection" the design's honesty stance (§0, §6, "must not be faked") is meant to prevent.

Let me note the one nuance for accuracy on severity: this requires a user to actually type "first/earliest N <time-or-shot-unit>". It is a real, reproducible mis-parse but it's an edge phrasing, not a core happy-path case, and it produces a wrong-but-bounded selection (caps to N cards) rather than a crash or data corruption. The suggested fix (negative lookahead) is correct and matches the existing bare-`Ns` refusal philosophy. I'll confirm the suggested fix regex behaves as claimed.

19. user (2026-06-20T16:37:40.949Z)

```

first 30 seconds    limit=3
first 5 minutes     no match (refused)
first 30 shots      limit=3
first 10            limit=10
first 10 longest    limit=10
first 30s          limit=3

```

20. assistant (2026-06-20T16:37:48.191Z)

The suggested fix as literally written has a bug worth noting (not part of the finding's truth, but relevant to fix quality): with `d+` greedy and the lookahead on the unit, `first 30 seconds` backtracks to capture `3` (lookahead at `0 seconds` passes since `0` isn't followed by a unit), yielding `limit=3` instead of refusing. A correct fix needs `(d+)\b` plus the unit lookahead, e.g.

```

\b(?:first|earliest)\s+(d+)\b(?:!\s*(?:s|secs?|seconds?|m|min|mins?|minutes?|shots?)\b)/.

```

I'll fold that correction into the fix field so the recommendation is actually sound.

The finding is **confirmed real**. Submitting.

21. user (2026-06-20T16:38:01.241Z)

Structured output provided successfully

22. assistant (2026-06-20T16:38:10.290Z)

Finding verified as **real** (is_real=true, major).

I reproduced `parseQuery("first 30 seconds")` against the actual compiled module (esbuild-bundled `query.ts`, run in Node), not just by eyeballing the regex:

- Output: `{filters:[], sort:{key:'start_time',dir:'asc'}, limit:30, recognized:true}`, pill `["oldest first","top 30"]`.
- Root cause confirmed at `C:/Users/avidu/Projects/khelsutra/web/src/lib/query.ts:168` ?
`/\b(?:first|earliest)\s+(\d+)\b/` matches "first 30" because the \b after \d+ falls between "30" and the space before "seconds", so the unit is never consulted.`
- Generalizes: `"first 5 minutes"?limit 5, "earliest 20 seconds"?limit 20, "first 30 shots"?limit 30`.
- Internal inconsistency confirmed: `"first 30s"` (no space) ? `recognized:false`, but `"first 30 seconds"` ? `limit:30`. Same intent, opposite outcomes ? and the latter contradicts the bare-`30s` refusal (`query.test.ts:63-66`) and the contract's honesty stance (`PER_RALLY_SELECTION.md §0/§6, D2/D5`).

One correction I folded into the fix field: the negative-lookahead exactly as written in the finding has its own bug ? greedy `\d+` backtracks so `"first 30 seconds"` captures `"3"` ? `limit=3` instead of refusing. The lookahead must follow a `\b` on the digits:
`/\b(?:first|earliest)\s+(\d+)\b(?:!\s*(?:s|secs?|seconds?|m|min|mins?|minutes?|shots?)\b)/``, which I verified refuses the unitful cases while still accepting `"first 10"` and `"first 10 longest"`.